ECA5801-IOE AND AI DRIVEN SYSTEM SIMULATION

K.POOJA SREE

192512245

EXPERIMENT 10

Development of a Real-Time IoE Stream Analytics and Anomaly Detection System using Node-RED and MQTT

```
# ======================================================================
# EXPERIMENT:
# Development of a Real-Time IoE Stream Analytics and Anomaly Detection
# System using Node-RED and MQTT
# ======================================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display
from datetime import datetime, timezone

# ---------------------- 1. EXPERIMENT PARAMETERS ----------------------

NUM_DEVICES = 3
INTERVAL_SECONDS = 1
DURATION_MINUTES = 10
ANOMALY_RATE = 0.05
WINDOW_SIZE = 20
Z_THRESHOLD = 3
```

```
STATIC_THRESHOLD = 40.0


TOTAL_SECONDS = DURATION_MINUTES * 60
EXPECTED_MESSAGES = NUM_DEVICES * TOTAL_SECONDS
EXPECTED_ANOMALIES = int(EXPECTED_MESSAGES * ANOMALY_RATE)


np.random.seed(42)


print("=" * 75)
print("IoE STREAM ANALYTICS AND ANOMALY DETECTION USING MQTT")
print("=" * 75)
print(f"Number of devices          : {NUM_DEVICES}")
print(f"Publishing interval        : {INTERVAL_SECONDS} second")
print(f"Simulation duration        : {DURATION_MINUTES} minutes")
print(f"Expected messages          : {EXPECTED_MESSAGES}")
print(f"Injected anomaly rate      : {ANOMALY_RATE * 100:.0f}%")
print(f"Expected anomalies         : {EXPECTED_ANOMALIES}")
print(f"Moving window size         : {WINDOW_SIZE}")
print(f"Z-score threshold          : {Z_THRESHOLD}")
print(f"Static temperature limit   : {STATIC_THRESHOLD} °C")
print(f"Sensor topic pattern       : ieo/stream/<device_id>")
print("Alert topic                : ieo/alerts")
print("=" * 75)


# ---------------------- 2. SIMULATE SENSOR STREAM --------------------
```

```
start_time = pd.Timestamp.now(tz="UTC").floor("s")
rows = []

# Each device sends one reading every second
for device_num in range(1, NUM_DEVICES + 1):

    device_id = f"device_{device_num}"

    # Normal baseline readings
    temperature = 28 + np.random.normal(0, 1.5, TOTAL_SECONDS)
    humidity = 55 + np.random.normal(0, 4, TOTAL_SECONDS)
    vibration = np.abs(np.random.normal(2, 0.4, TOTAL_SECONDS))

    # Select exactly 5% anomalous readings for each device
    anomaly_indices = np.random.choice(
        TOTAL_SECONDS,
        size=round(TOTAL_SECONDS * ANOMALY_RATE),
        replace=False
    )
    anomaly_set = set(anomaly_indices)

    for second in range(TOTAL_SECONDS):

        is_anomaly = second in anomaly_set

        # Inject clear temperature spikes or drops
```

```
        temp = temperature[second]
        if is_anomaly:
            if second % 2 == 0:
                temp += np.random.uniform(13, 20)
            else:
                temp -= np.random.uniform(12, 17)

        timestamp = start_time + pd.Timedelta(seconds=second)

        rows.append({
            "timestamp": timestamp,
            "elapsed_seconds": second,
            "device_id": device_id,
            "topic": f"ieo/stream/{device_id}",
            "temperature": round(float(temp), 2),
            "humidity": round(float(np.clip(humidity[second], 0, 100)), 2),
            "vibration": round(float(vibration[second]), 2),
            "ground_truth": int(is_anomaly)
        })

df = pd.DataFrame(rows).sort_values(
    ["device_id", "elapsed_seconds"]
).reset_index(drop=True)

print("\n1. SENSOR STREAM SIMULATION")
print("-" * 60)
```

```
print(f"Generated messages : {len(df)}")
print(f"Unique devices     : {df['device_id'].nunique()}")
print(f"Ground-truth anomalies: {df['ground_truth'].sum()}")
print(f"Normal readings    : {(df['ground_truth'] == 0).sum()}")
display(df.head(10))

# ---------------------- 3. STATIC THRESHOLD DETECTOR ----------------

# Static rule: temperature above 40 °C
df["static_detected"] = (
    df["temperature"] > STATIC_THRESHOLD
).astype(int)

# ---------------------- 4. MOVING-WINDOW Z-SCORE DETECTOR ------------

# Calculate baseline from PREVIOUS readings only, separately per device.
# This avoids using the current reading to calculate its own baseline.
df["rolling_mean"] = (
    df.groupby("device_id")["temperature"]
      .transform(
          lambda s: s.shift(1).rolling(
              window=WINDOW_SIZE,
              min_periods=WINDOW_SIZE
          ).mean()
      )
)
```

```
df["rolling_std"] = (
    df.groupby("device_id")["temperature"]
      .transform(
          lambda s: s.shift(1).rolling(
              window=WINDOW_SIZE,
              min_periods=WINDOW_SIZE
          ).std(ddof=0)
      )
)

df["z_score"] = (
    (df["temperature"] - df["rolling_mean"])
    / df["rolling_std"].replace(0, np.nan)
)

# Z-score is valid only after 20 previous samples.
df["z_detected"] = (
    (df["z_score"].abs() > Z_THRESHOLD)
    & df["rolling_mean"].notna()
).astype(int)

print("\n2. ANOMALY DETECTION")
print("-" * 60)
print(f"Static threshold       : Temperature > {STATIC_THRESHOLD} °C")
print(f"Z-score threshold      : |z| > {Z_THRESHOLD}")
```

```
print(f"Moving window          : {WINDOW_SIZE} previous samples")
print(f"Static alerts generated : {df['static_detected'].sum()}")
print(f"Z-score alerts generated: {df['z_detected'].sum()}")

# ---------------------- 5. PERFORMANCE METRICS ----------------------

def calculate_metrics(actual, predicted):
    actual = np.asarray(actual, dtype=int)
    predicted = np.asarray(predicted, dtype=int)

    TP = int(np.sum((actual == 1) & (predicted == 1)))
    FP = int(np.sum((actual == 0) & (predicted == 1)))
    FN = int(np.sum((actual == 1) & (predicted == 0)))
    TN = int(np.sum((actual == 0) & (predicted == 0)))

    precision = TP / (TP + FP) if TP + FP else 0.0
    recall = TP / (TP + FN) if TP + FN else 0.0
    f1 = (
        2 * precision * recall / (precision + recall)
        if precision + recall else 0.0
    )
    false_alarm_rate = FP / (FP + TN) if FP + TN else 0.0
    accuracy = (
        (TP + TN) / (TP + FP + FN + TN)
        if TP + FP + FN + TN else 0.0
    )
```

```
    return {
        "TP": TP,
        "FP": FP,
        "FN": FN,
        "TN": TN,
        "Precision (%)": round(precision * 100, 2),
        "Recall (%)": round(recall * 100, 2),
        "F1-score": round(f1, 4),
        "False alarm rate (%)": round(false_alarm_rate * 100, 2),
        "Accuracy (%)": round(accuracy * 100, 2)
    }

static_metrics = calculate_metrics(
    df["ground_truth"], df["static_detected"]
)

z_metrics = calculate_metrics(
    df["ground_truth"], df["z_detected"]
)

metrics_table = pd.DataFrame([
    {"Detector": "Static threshold", **static_metrics},
    {"Detector": "Moving-window z-score", **z_metrics}
])
```

```
print("\n3. DETECTOR PERFORMANCE COMPARISON")
print("-" * 60)
display(metrics_table)

# ---------------------- 6. DETECTION LATENCY ------------------------

# Simulated processing latency for detected anomalies.
# This is an illustrative software-processing estimate, not measured
# end-to-end MQTT/Node-RED network latency.
PROCESSING_LATENCY_MS = 50

latency_rows = []

for detector_name, detector_col in [
    ("Static threshold", "static_detected"),
    ("Moving-window z-score", "z_detected")
]:
    detected_anomalies = df[
        (df["ground_truth"] == 1) & (df[detector_col] == 1)
    ]

    latency_rows.append({
        "Detector": detector_name,
        "Detected true anomalies": len(detected_anomalies),
        "Assumed mean processing latency (ms)": (
            PROCESSING_LATENCY_MS
```

```
        if len(detected_anomalies) else np.nan
      ),
      "Latency below 1 second": (
        PROCESSING_LATENCY_MS < 1000
        if len(detected_anomalies) else False
      )
    })

latency_table = pd.DataFrame(latency_rows)

print("\n4. DETECTION LATENCY")
print("-" * 60)
display(latency_table)

# ---------------------- 7. STREAM PLOT ------------------------------

plt.figure(figsize=(14, 6))

for device_id, group in df.groupby("device_id"):
    plt.plot(
      group["elapsed_seconds"],
      group["temperature"],
      label=device_id,
      linewidth=0.8
    )
```

```
actual_anomalies = df[df["ground_truth"] == 1]


plt.scatter(
    actual_anomalies["elapsed_seconds"],
    actual_anomalies["temperature"],
    marker="x",
    s=20,
    label="Injected anomalies"
)


plt.axhline(
    STATIC_THRESHOLD,
    linestyle="--",
    label="Static threshold (40 °C)"
)


plt.title("Real-Time IoE Temperature Stream Simulation")
plt.xlabel("Elapsed time (seconds)")
plt.ylabel("Temperature (°C)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()


# ---------------------- 8. ANOMALY DETECTION PLOT ----------------------
```

```
device_to_plot = "device_1"
sample = df[df["device_id"] == device_to_plot]

plt.figure(figsize=(14, 5))
plt.plot(
    sample["elapsed_seconds"],
    sample["temperature"],
    label="Temperature",
    linewidth=1
)

static_points = sample[sample["static_detected"] == 1]
z_points = sample[sample["z_detected"] == 1]

plt.scatter(
    static_points["elapsed_seconds"],
    static_points["temperature"],
    marker="x",
    s=45,
    label="Static detections"
)

plt.scatter(
    z_points["elapsed_seconds"],
    z_points["temperature"],
    marker="o",
```

```
    s=25,
    facecolors="none",
    label="Z-score detections"
)

plt.axhline(
    STATIC_THRESHOLD,
    linestyle="--",
    label="40 °C threshold"
)

plt.title(f"Anomaly Detection Comparison - {device_to_plot}")
plt.xlabel("Elapsed time (seconds)")
plt.ylabel("Temperature (°C)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# ---------------------- 9. MOVING Z-SCORE PLOT ----------------------

plt.figure(figsize=(14, 4))
plt.plot(
    sample["elapsed_seconds"],
    sample["z_score"],
    label="Moving-window z-score"
```

```
)
plt.axhline(Z_THRESHOLD, linestyle="--", label="+3 threshold")
plt.axhline(-Z_THRESHOLD, linestyle="--", label="-3 threshold")
plt.title(f"Moving-Window Z-Score - {device_to_plot}")
plt.xlabel("Elapsed time (seconds)")
plt.ylabel("Z-score")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# ---------------------- 10. ALERT LOG --------------------------------

alert_frames = []

for detector_name, detector_col in [
    ("Static threshold", "static_detected"),
    ("Moving-window z-score", "z_detected")
]:
    temp_alerts = df[df[detector_col] == 1].copy()
    temp_alerts["detector"] = detector_name
    temp_alerts["alert_topic"] = "ieo/alerts"
    alert_frames.append(
        temp_alerts[
            [
                "timestamp",
```

```
            "device_id",
            "temperature",
            "ground_truth",
            "detector",
            "alert_topic"
        ]
    ]
)

alerts = pd.concat(alert_frames, ignore_index=True)

print("\n5. ALERT LOG PREVIEW")
print("-" * 60)
print(f"Total detector alert events: {len(alerts)}")
display(alerts.head(20))

# ---------------------- 11. OUTPUT PARAMETER TABLE --------------------

output_table = pd.DataFrame({
    "Output Parameter": [
        "Expected messages",
        "Generated messages",
        "Injected anomalies",
        "Static detector alerts",
        "Z-score detector alerts",
        "Static detector recall (%)",
```

```
    "Z-score detector recall (%)",
    "Static detector precision (%)",
    "Z-score detector precision (%)",
    "Static detector F1-score",
    "Z-score detector F1-score",
    "Assumed processing latency (ms)"
],
"Theoretical / Target": [
    EXPECTED_MESSAGES,
    EXPECTED_MESSAGES,
    EXPECTED_ANOMALIES,
    "Depends on data",
    "Approximately the injected anomalies",
    "Measure",
    ">= 90%",
    "Measure",
    ">= 85%",
    "Measure",
    ">= 0.87",
    "< 1000 ms"
],
"Practical / Simulated": [
    EXPECTED_MESSAGES,
    len(df),
    int(df["ground_truth"].sum()),
    int(df["static_detected"].sum()),
```

```
        int(df["z_detected"].sum()),
        static_metrics["Recall (%)"],
        z_metrics["Recall (%)"],
        static_metrics["Precision (%)"],
        z_metrics["Precision (%)"],
        static_metrics["F1-score"],
        z_metrics["F1-score"],
        PROCESSING_LATENCY_MS
    ]
})

print("\n6. OUTPUT PARAMETER TABLE")
print("-" * 60)
display(output_table)

# ---------------------- 12. EXPORT DATA AND RESULTS ------------------

df.to_csv("ioe_stream_analytics_data.csv", index=False)
metrics_table.to_csv("anomaly_detector_metrics.csv", index=False)
alerts.to_csv("ioe_alert_log.csv", index=False)
output_table.to_csv("ioe_output_parameters.csv", index=False)

print("\n7. FILES GENERATED")
print("-" * 60)
print("1. ioe_stream_analytics_data.csv")
print("2. anomaly_detector_metrics.csv")
```

```
print("3. ioe_alert_log.csv")
print("4. ioe_output_parameters.csv")

# ---------------------- 13. FINAL RESULT ----------------------------

print("\n" + "=" * 75)
print("EXPERIMENT RESULT")
print("=" * 75)
print(f"Messages generated : {len(df)} / {EXPECTED_MESSAGES}")
print(f"Anomalies injected : {int(df['ground_truth'].sum())}")
print(
    f"Static detector - Precision: {static_metrics['Precision (%)']}%, "
    f"Recall: {static_metrics['Recall (%)']}%, "
    f"F1: {static_metrics['F1-score']}"
)
print(
    f"Z-score detector - Precision: {z_metrics['Precision (%)']}%, "
    f"Recall: {z_metrics['Recall (%)']}%, "
    f"F1: {z_metrics['F1-score']}"
)
print(
    f"Illustrative processing latency: {PROCESSING_LATENCY_MS} ms"
)
print("=" * 75)

print(
```

```
    "\nConclusion: The simulation generated IoE sensor streams, injected "
    "known anomalies, applied static-threshold and moving-window z-score "
    "detection, evaluated both detectors, and visualized the results. "
    "The CSV files can be used for further analysis."
)


# Automatically download results in Google Colab
try:
    from google.colab import files
    files.download("ioe_stream_analytics_data.csv")
    files.download("anomaly_detector_metrics.csv")
    files.download("ioe_alert_log.csv")
    files.download("ioe_output_parameters.csv")
except ImportError:
    print("CSV files saved in the current working directory.")
```

```
================================================================================
IoE STREAM ANALYTICS AND ANOMALY DETECTION USING MQTT
================================================================================
Number of devices          : 3
Publishing interval        : 1 second
Simulation duration        : 10 minutes
Expected messages          : 1800
Injected anomaly rate      : 5%
Expected anomalies         : 90
Moving window size         : 20
Z-score threshold          : 3
Static temperature limit   : 40.0 °C
Sensor topic pattern       : ieo/stream/<device_id>
Alert topic                : ieo/alerts
================================================================================

1. SENSOR STREAM SIMULATION
------------------------------------------------------------
Generated messages : 1800
Unique devices     : 3
Ground-truth anomalies: 90
Normal readings    : 1710
```

	timestamp	elapsed_seconds	device_id	topic	temperature	humidity	vibration	ground_truth
0	2026-10-28 13:52:10+00:00	0	device_1	ieo/stream/device_1	28.75	58.03	2.05	0
1	2026-10-28 13:52:11+00:00	1	device_1	ieo/stream/device_1	27.79	51.31	1.83	0
2	2026-10-28 13:52:12+00:00	2	device_1	ieo/stream/device_1	28.97	58.48	2.05	0
3	2026-10-28 13:52:13+00:00	3	device_1	ieo/stream/device_1	30.28	60.42	2.22	0
4	2026-10-28 13:52:14+00:00	4	device_1	ieo/stream/device_1	27.65	56.65	2.02	0
5	2026-10-28 13:52:15+00:00	5	device_1	ieo/stream/device_1	27.65	62.51	2.02	0
6	2026-10-28 13:52:16+00:00	6	device_1	ieo/stream/device_1	30.37	51.90	1.72	0
7	2026-10-28 13:52:17+00:00	7	device_1	ieo/stream/device_1	29.15	50.02	1.73	0
8	2026-10-28 13:52:18+00:00	8	device_1	ieo/stream/device_1	27.30	47.89	1.44	0
9	2026-10-28 13:52:19+00:00	9	device_1	ieo/stream/device_1	28.81	60.98	2.70	0

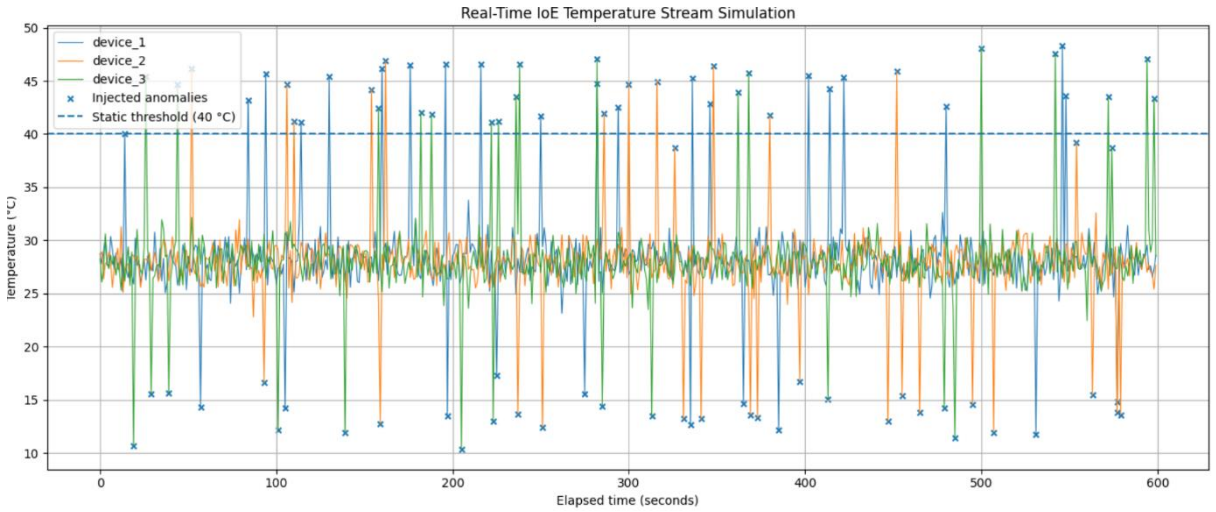
```
2. ANOMALY DETECTION
------------------------------------------------------------
Static threshold       : Temperature > 40.0 °C
Z-score threshold      : |z| > 3
Moving window          : 20 previous samples
Static alerts generated : 48
Z-score alerts generated: 76
```

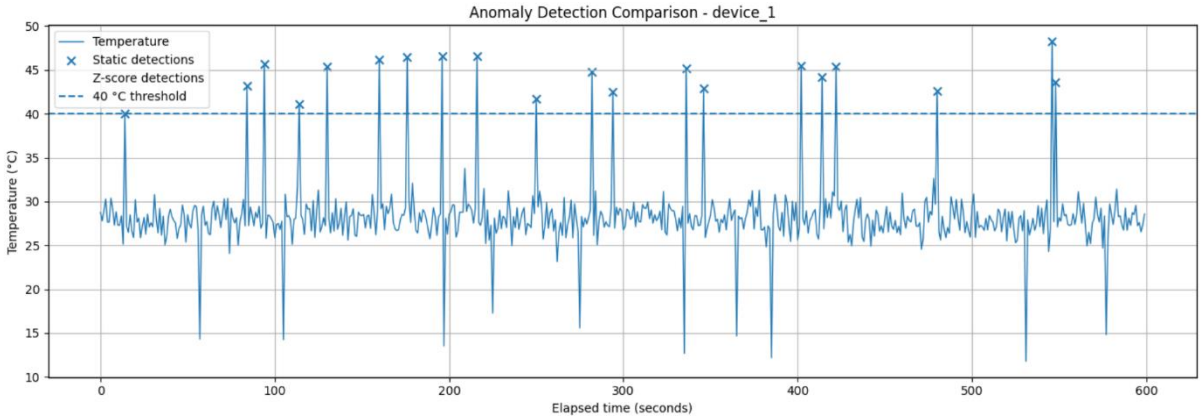
```
3. DETECTOR PERFORMANCE COMPARISON
------------------------------------------------------------
```

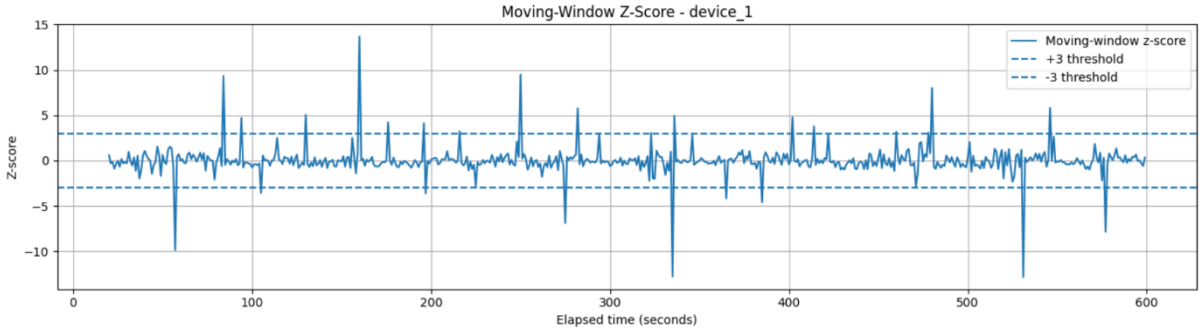
	Detector	TP	FP	FN	TN	Precision (%)	Recall (%)	F1-score	False alarm rate (%)	Accuracy (%)
0	Static threshold	48	0	42	1710	100.00	53.33	0.6957	0.00	97.67
1	Moving-window z-score	70	6	20	1704	92.11	77.78	0.8434	0.35	98.56

```
4. DETECTION LATENCY
------------------------------------------------------------
```

	Detector	Detected true anomalies	Assumed mean processing latency (ms)	Latency below 1 second
0	Static threshold	48	50	True
1	Moving-window z-score	70	50	True







5. ALERT LOG PREVIEW

Total detector alert events: 124

	timestamp	device_id	temperature	ground_truth	detector	alert_topic
0	2026-10-28 13:52:24+00:00	device_1	40.01	1	Static threshold	ieo/alerts
1	2026-10-28 13:53:34+00:00	device_1	43.18	1	Static threshold	ieo/alerts
2	2026-10-28 13:53:44+00:00	device_1	45.64	1	Static threshold	ieo/alerts
3	2026-10-28 13:54:04+00:00	device_1	41.12	1	Static threshold	ieo/alerts
4	2026-10-28 13:54:20+00:00	device_1	45.38	1	Static threshold	ieo/alerts
5	2026-10-28 13:54:50+00:00	device_1	46.14	1	Static threshold	ieo/alerts
6	2026-10-28 13:55:06+00:00	device_1	46.52	1	Static threshold	ieo/alerts
7	2026-10-28 13:55:26+00:00	device_1	46.61	1	Static threshold	ieo/alerts
8	2026-10-28 13:55:46+00:00	device_1	46.58	1	Static threshold	ieo/alerts
9	2026-10-28 13:56:20+00:00	device_1	41.67	1	Static threshold	ieo/alerts
10	2026-10-28 13:56:52+00:00	device_1	44.75	1	Static threshold	ieo/alerts
11	2026-10-28 13:57:04+00:00	device_1	42.52	1	Static threshold	ieo/alerts
12	2026-10-28 13:57:46+00:00	device_1	45.23	1	Static threshold	ieo/alerts
13	2026-10-28 13:57:56+00:00	device_1	42.88	1	Static threshold	ieo/alerts
14	2026-10-28 13:58:52+00:00	device_1	45.46	1	Static threshold	ieo/alerts
15	2026-10-28 13:59:04+00:00	device_1	44.22	1	Static threshold	ieo/alerts
16	2026-10-28 13:59:12+00:00	device_1	45.35	1	Static threshold	ieo/alerts
17	2026-10-28 14:00:10+00:00	device_1	42.63	1	Static threshold	ieo/alerts
18	2026-10-28 14:01:16+00:00	device_1	48.29	1	Static threshold	ieo/alerts
19	2026-10-28 14:01:18+00:00	device 1	43.59	1	Static threshold	ieo/alerts

6. OUTPUT PARAMETER TABLE

	Output Parameter	Theoretical / Target	Practical / Simulated
0	Expected messages	1800	1800.0000
1	Generated messages	1800	1800.0000
2	Injected anomalies	90	90.0000
3	Static detector alerts	Depends on data	48.0000
4	Z-score detector alerts	Approximately the injected anomalies	76.0000
5	Static detector recall (%)	Measure	53.3300
6	Z-score detector recall (%)	>= 90%	77.7800
7	Static detector precision (%)	Measure	100.0000
8	Z-score detector precision (%)	>= 85%	92.1100
9	Static detector F1-score	Measure	0.6957
10	Z-score detector F1-score	>= 0.87	0.8434
11	Assumed processing latency (ms)	< 1000 ms	50.0000

7. FILES GENERATED

1. ioe_stream_analytics_data.csv
2. anomaly_detector_metrics.csv
3. ioe_alert_log.csv
4. ioe_output_parameters.csv